Tutorial Sheet — TD 2 / Correction Guidelines

→ Solution – Exercise 1

Here $\mathbf{a = 2}$, $\mathbf{b = 2}$, $\mathbf{f(n) = n}$. We compare $\mathbf{f(n)}$ with $\mathbf{n^{(\log_b a)} = n^{(\log_2 2)} = n^1 = n}$.

Since $\mathbf{f(n) = \Theta(n^{(\log_b a)})}$, we are in Case 2 of the Master Theorem, which gives:

$\mathbf{T(n) = \Theta(n \log n)}$

→ Solution – Exercise 2

Here $\mathbf{a = 3}$, $\mathbf{b = 4}$, $\mathbf{f(n) = n^2}$. We compare $\mathbf{f(n)}$ with $\mathbf{n^{(\log_b a)} = n^{(\log_4 3)} \approx n^{0.79}}$.

Since $\mathbf{f(n) = n^2}$ grows polynomially faster than $\mathbf{n^{0.79}}$ (i.e. $\mathbf{f(n) = \Omega(n^{(\log_b a + \varepsilon)})}$ for $\boldsymbol{\varepsilon \approx 1.2}$), we check the regularity condition: $\mathbf{a \cdot f(n/b) = 3(n/4)^2 = 3n^2/16 \leq c \cdot n^2}$ for $\mathbf{c = 3/16 < 1}$. The condition holds, so we are in Case 3:

$\mathbf{T(n) = \Theta(n^2)}$

→ Model Answer (Corrigé Type) [Exercise 3]

1) Recurrence: $\mathbf{T(n) = T(n-1) + T(n-2) + \Theta(1)}$, with $\mathbf{T(0) = T(1) = \Theta(1)}$. This is not of the form $\mathbf{T(n) = aT(n/b) + f(n)}$ required by the Master Theorem: the two subproblems are of sizes $\mathbf{n-1}$ and $\mathbf{n-2}$ (a subtractive, not divisive, recurrence), so the theorem's hypotheses simply do not apply.
2) Since $\mathbf{T(n-1)}$ and $\mathbf{T(n-2)}$ are both bounded above by $\mathbf{T(n-1)}$ (as $\mathbf{T}$ is increasing), we get $\mathbf{T(n) \leq 2T(n-1) + \Theta(1)}$. Unrolling this bound gives a recursion tree of depth $\mathbf{n}$ where the work at most doubles at each level, so $\mathbf{T(n) = O(2^n)}$. More precisely, unrolling $\mathbf{k}$ levels gives $\mathbf{T(n) \leq 2^k T(n-k) + (2^k - 1) \cdot \Theta(1)}$; taking $\mathbf{k = n}$ shows $\mathbf{T(n) = O(2^n)}$.
3) Iterative version: maintain two variables $\mathbf{a = fib(i-1)}$, $\mathbf{b = fib(i)}$ and update $\mathbf{(a, b) \leftarrow (b, a+b)}$ in a single loop from $\mathbf{i = 2}$ to $\mathbf{n}$, returning $\mathbf{b}$. Each of the $\mathbf{n}$ loop iterations does $\mathbf{O(1)}$ work, giving $\mathbf{\Theta(n)}$ total time. It avoids the exponential blow-up because each Fibonacci value is computed exactly once and reused, whereas the naive recursion recomputes the same subproblems (e.g. $\mathbf{fib(n-2)}$) many times over.

Common mistakes to avoid:

- Trying to force-fit $\mathbf{T(n) = T(n-1) + T(n-2)}$ into the Master Theorem's $\mathbf{T(n) = aT(n/b) + f(n)}$ template — the theorem simply does not cover subtractive recurrences.
- Believing the $\mathbf{O(2^n)}$ bound from the recursion tree is tight; the true bound is $\mathbf{\Theta(\varphi^n)}$ with $\boldsymbol{\varphi \approx 1.618}$ (the golden ratio), which is a strictly smaller exponential, the recursion-tree argument above only gives an easy upper bound, not the exact order.